

Desenvolvimento de Sistemas Web - Atividade de JavaScript

Aluno: Kellyson Medeiros

Professor: Irlan Arley Targino Moreira

Data: 4 de setembro

Questão 1. Escreva um programa que, a partir de uma duração em segundos, calcule e exiba a quantas horas, minutos e segundos ela equivale (ex.: 3665 → 1h 1min 5s).

```
function converterSegundos(totalSegundos) {  
    const horas = Math.floor(totalSegundos / 3600);  
    const minutos = Math.floor((totalSegundos % 3600) / 60);  
    const segundos = totalSegundos % 60;  
    return `${horas}h ${minutos}min ${segundos}s`;  
}
```

```
console.log(converterSegundos(3665));
```

Saída no console:

1h 1min 5s

Questão 2. Dada uma temperatura em Celsius, calcule e exiba o equivalente em Fahrenheit e em Kelvin.

```
function converterTemperatura(celsius) {  
    const fahrenheit = (celsius * 9) / 5 + 32;  
    const kelvin = celsius + 273.15;  
  
    console.log(`${celsius}°C equivale a ${fahrenheit.toFixed(2)}°F e ${kelvin.toFixed(2)}K`);  
}
```

```
converterTemperatura(25);
```

Saída no console:

25°C equivale a 77.00°F e 298.15K

Questão 3. Escreva um programa que recebe o preço de um produto e a quantidade comprada, calcula o subtotal, aplica um desconto progressivo (5% acima de R\$100, 10% acima de R\$300) e exibe um resumo formatado do pedido usando template literals.

```
function resumoPedido(preco, quantidade) {  
    const subtotal = preco * quantidade;  
    let desconto = 0;  
  
    if (subtotal > 300) {  
        desconto = subtotal * 0.10;  
    } else if (subtotal > 100) {  
        desconto = subtotal * 0.05;  
    }  
  
    const total = subtotal - desconto;
```

```

        return `Preço unitário: R$ ${preco.toFixed(2)}
Quantidade: ${quantidade}
Subtotal: R$ ${subtotal.toFixed(2)}
Desconto: R$ ${desconto.toFixed(2)}
Total a pagar: R$ ${total.toFixed(2)}`;
    }

```

```
console.log(resumoPedido(50, 4));
```

Saída no console:

```

Preço unitário: R$ 50.00
Quantidade: 4
Subtotal: R$ 200.00
Desconto: R$ 10.00
Total a pagar: R$ 190.00

```

Questão 4. Dado o ano de nascimento de uma pessoa, calcule a idade e determine, combinando operadores lógicos e de comparação, se ela já pode votar, se o voto é obrigatório e se ela ainda está isenta do serviço militar.

```

function verificarSituacao(anoNascimento) {
    const anoAtual = new Date().getFullYear();
    const idade = anoAtual - anoNascimento;

    const podeVotar = idade >= 16;
    const votoObrigatorio = idade >= 18 && idade < 70;
    const isentoServicoMilitar = idade < 18;

    console.log(`Idade: ${idade}`);
    console.log(`Pode votar: ${podeVotar ? "Sim" : "Não"}`);
    console.log(`Voto obrigatório: ${votoObrigatorio ? "Sim" : "Não"}`);
    console.log(`Isento do serviço militar: ${isentoServicoMilitar ? "Sim" : "Não"}`);
}

```

```
verificarSituacao(2008);
```

Saída no console:

```

Idade: 18
Pode votar: Sim
Voto obrigatório: Sim
Isento do serviço militar: Não

```

Questão 5. Implemente uma calculadora: uma função recebe dois números e um operador ('+', '-', '*', '/') e retorna o resultado. Trate operador inválido e divisão por zero. Use switch/case.

```

function calculadora(a, b, operador) {
    switch (operador) {

```

```

    case '+':
        return a + b;
    case '-':
        return a - b;
    case '*':
        return a * b;
    case '/':
        if (b === 0) {
            return "Erro: divisão por zero";
        }
        return a / b;
    default:
        return "Erro: operador inválido";
}
}

```

```

console.log(calculadora(10, 2, '+'));
console.log(calculadora(10, 0, '/'));
console.log(calculadora(10, 2, '%'));

```

Saída no console:

12

Erro: divisão por zero

Erro: operador inválido

Questão 6. Escreva uma função que recebe peso e altura e retorna a classificação de IMC da pessoa nas faixas usuais (abaixo do peso, normal, sobrepeso, obesidade).

```

function classificarIMC(peso, altura) {
    const imc = peso / (altura * altura);

    if (imc < 18.5) {
        return "Abaixo do peso";
    } else if (imc < 25) {
        return "Normal";
    } else if (imc < 30) {
        return "Sobrepeso";
    } else {
        return "Obesidade";
    }
}

```

```

console.log(classificarIMC(70, 1.75));

```

Saída no console:

Normal

Questão 7. Implemente Pedra, Papel e Tesoura: uma função recebe as jogadas de dois jogadores e retorna quem venceu, ou empate. Evite repetir a mesma comparação três vezes.

```
function pedraPapelTesoura(jogador1, jogador2) {
  const j1 = jogador1.toLowerCase();
  const j2 = jogador2.toLowerCase();

  if (j1 === j2) {
    return "Empate";
  }

  const vitorias = {
    pedra: 'tesoura',
    papel: 'pedra',
    tesoura: 'papel'
  };

  if (vitorias[j1] === j2) {
    return "Jogador 1 venceu";
  }

  return "Jogador 2 venceu";
}

console.log(pedraPapelTesoura("pedra", "tesoura"));
console.log(pedraPapelTesoura("papel", "papel"));
```

Saída no console:

Jogador 1 venceu

Empate

Questão 8. Escreva uma função que recebe uma string e retorna se ela é um palíndromo, ignorando maiúsculas/minúsculas e espaços.

```
function ehPalindromo(texto) {
  const limpo = texto.toLowerCase().replace(/\s+/g, '');
  const invertido = limpo.split('').reverse().join('');
  return limpo === invertido;
}

console.log(ehPalindromo("A base do teto desaba"));
console.log(ehPalindromo("javascript"));
```

Saída no console:

true

false

Questão 9. Escreva uma função que recebe um array de notas de uma turma e retorna um objeto com a média, a maior e a menor nota.

```
function analisarNotas(notas) {
  if (!notas || notas.length === 0) return null;

  const soma = notas.reduce((acc, nota) => acc + nota, 0);
```

```
const media = soma / notas.length;
const maior = Math.max(...notas);
const menor = Math.min(...notas);

return {
  media: Number(media.toFixed(2)),
  maior,
  menor
};
}

console.log(analisarNotas([7.5, 8.0, 9.5, 6.0, 10.0]));
Saída no console:
{ media: 8.2, maior: 10, menor: 6 }
```

Questão 10. Escreva uma função que calcule o n-ésimo termo da sequência de Fibonacci (iterativa ou recursiva, à sua escolha).

```
function fibonacci(n) {
  if (n <= 0) return 0;
  if (n === 1) return 1;

  let anterior = 0;
  let atual = 1;

  for (let i = 2; i <= n; i++) {
    const proximo = anterior + atual;
    anterior = atual;
    atual = proximo;
  }

  return atual;
}

console.log(fibonacci(7));
Saída no console:
13
```

Questão 11. Escreva uma função de validação de senha que retorna quais regras a senha não cumpre, entre: mínimo de 8 caracteres, ao menos uma letra maiúscula, ao menos um número.

```
function validarSenha(senha) {
  const pendencias = [];

  if (senha.length < 8) {
    pendencias.push("Mínimo de 8 caracteres");
  }
  if (!/[A-Z]/.test(senha)) {
```

```

        pendencias.push("Ao menos uma letra maiúscula");
    }
    if (!/[0-9]/.test(senha)) {
        pendencias.push("Ao menos um número");
    }

    return pendencias;
}

```

```

console.log(validarSenha("abc"));
console.log(validarSenha("Senha123"));

```

Saída no console:

```

[ 'Mínimo de 8 caracteres', 'Ao menos uma letra maiúscula', 'Ao menos um
número' ]
[]

```

Questão 12. Dado um array de produtos em estoque (nome, preço, quantidade), escreva funções que calculem o valor total do estoque, encontrem o produto mais caro e listem os produtos com quantidade abaixo de um mínimo informado.

```

function calcularValorTotalEstoque(produtos) {
    return produtos.reduce((total, p) => total + p.preco * p.quantidade, 0);
}

function encontrarProdutoMaisCaro(produtos) {
    return produtos.reduce((maisCaro, p) => (p.preco > maisCaro.preco ? p :
maisCaro), produtos[0]);
}

function listarAbaixoDoMinimo(produtos, minimo) {
    return produtos.filter(p => p.quantidade < minimo);
}

const produtos = [
    { nome: "Teclado", preco: 250, quantidade: 5 },
    { nome: "Mouse", preco: 120, quantidade: 2 },
    { nome: "Monitor", preco: 900, quantidade: 1 }
];

console.log("Total:", calcularValorTotalEstoque(produtos));
console.log("Mais caro:", encontrarProdutoMaisCaro(produtos));
console.log("Abaixo de 3 unidades:", listarAbaixoDoMinimo(produtos, 3));

```

Saída no console:

```

Total: 2390
Mais caro: { nome: 'Monitor', preco: 900, quantidade: 1 }
Abaixo de 3 unidades: [
  { nome: 'Mouse', preco: 120, quantidade: 2 },

```

```
    { nome: 'Monitor', preco: 900, quantidade: 1 }  
  ]
```

Questão 13. Modele uma agenda de contatos (array de objetos com nome, telefone e categoria) e escreva funções para adicionar um contato, remover pelo nome e listar os contatos de uma categoria.

Dado um array de números com valores repetidos, escreva uma função que retorna um novo array só com os valores únicos.

```
let agenda = [  
  { nome: "Carlos", telefone: "9999-1111", categoria: "Trabalho" },  
  { nome: "Mariana", telefone: "9888-2222", categoria: "Amigos" }  
];  
  
function adicionarContato(agenda, nome, telefone, categoria) {  
  agenda.push({ nome, telefone, categoria });  
  return agenda;  
}  
  
function removerContatoPorNome(agenda, nome) {  
  const index = agenda.findIndex(c => c.nome.toLowerCase() ===  
nome.toLowerCase());  
  if (index !== -1) {  
    agenda.splice(index, 1);  
  }  
  return agenda;  
}  
  
function listarPorCategoria(agenda, categoria) {  
  return agenda.filter(c => c.categoria.toLowerCase() ===  
categoria.toLowerCase());  
}  
  
function obterValoresUnicos(array) {  
  return [...new Set(array)];  
}  
  
console.log(listarPorCategoria(agenda, "Trabalho"));  
console.log(obterValoresUnicos([1, 2, 2, 3, 4, 4, 5]));  
Saída no console:  
[ { nome: 'Carlos', telefone: '9999-1111', categoria: 'Trabalho' } ]  
[ 1, 2, 3, 4, 5 ]
```

Questão 14. Modele uma classe Produto com nome, preço e quantidade em estoque. Adicione métodos para aplicar um desconto percentual e para informar se o produto está disponível (quantidade > 0). Instancie ao menos três produtos e demonstre os métodos.

```

class Produto {
    constructor(nome, preco, quantidade) {
        this.nome = nome;
        this.preco = preco;
        this.quantidade = quantidade;
    }

    aplicarDesconto(percentual) {
        this.preco = this.preco * (1 - percentual / 100);
    }

    estaDisponivel() {
        return this.quantidade > 0;
    }
}

const p1 = new Produto("Notebook", 3500, 4);
const p2 = new Produto("Fone Bluetooth", 150, 0);
const p3 = new Produto("Webcam", 200, 8);

p1.aplicarDesconto(10);

console.log(p1.nome, "- R$", p1.preco, "| Disponível:", p1.estaDisponivel());
console.log(p2.nome, "- R$", p2.preco, "| Disponível:", p2.estaDisponivel());
console.log(p3.nome, "- R$", p3.preco, "| Disponível:", p3.estaDisponivel());

```

Saída no console:

```

Notebook - R$ 3150 | Disponível: true
Fone Bluetooth - R$ 150 | Disponível: false
Webcam - R$ 200 | Disponível: true

```

Questão 15. Modele uma classe **ContaBancaria** com titular e saldo. Implemente os métodos **depositar(valor)** e **sacar(valor)** — impedindo saldo negativo — e um método **extrato()** que retorna um resumo formatado da conta. Simule algumas operações entre duas contas.

```

class ContaBancaria {
    constructor(titular, saldo = 0) {
        this.titular = titular;
        this.saldo = saldo;
    }

    depositar(valor) {
        if (valor > 0) {
            this.saldo += valor;
            return true;
        }
        return false;
    }
}

```

```

    sacar(valor) {
      if (valor > 0 && valor <= this.saldo) {
        this.saldo -= valor;
        return true;
      }
      return false;
    }

    extrato() {
      return `Titular: ${this.titular} | Saldo: R$ ${this.saldo.toFixed(2)}
`;
    }
  }
}

```

```

const conta1 = new ContaBancaria("Kellyson", 500);
const conta2 = new ContaBancaria("Maria", 200);

```

```

conta1.depositar(150);
conta1.sacar(100);
conta2.depositar(50);
conta2.sacar(400);

```

```

console.log(conta1.extrato());
console.log(conta2.extrato());

```

Saída no console:

Titular: Kellyson | Saldo: R\$ 550.00

Titular: Maria | Saldo: R\$ 250.00

Questão 16. Modele uma classe Retangulo com base e altura, com métodos para calcular área e perímetro.

```

class Retangulo {
  constructor(base, altura) {
    this.base = base;
    this.altura = altura;
  }

  calcularArea() {
    return this.base * this.altura;
  }

  calcularPerimetro() {
    return 2 * (this.base + this.altura);
  }
}

const ret = new Retangulo(10, 5);
console.log("Área:", ret.calcularArea());
console.log("Perímetro:", ret.calcularPerimetro());

```

Saída no console:

Área: 50

Perímetro: 30

Questão 17. Escolha um dos exercícios de array de objetos que você já resolveu (agenda de contatos, estoque ou carrinho de compras) e reescreva-o usando uma classe, com os dados como propriedades de instância e as operações como métodos. Compare as duas versões: o que ficou mais organizado?

```
class Estoque {
  constructor(produtosIniciais = []) {
    this.produtos = produtosIniciais;
  }

  adicionarProduto(produto) {
    this.produtos.push(produto);
  }

  calcularValorTotal() {
    return this.produtos.reduce((total, p) => total + p.preco *
p.quantidade, 0);
  }

  obterProdutoMaisCaro() {
    if (this.produtos.length === 0) return null;
    return this.produtos.reduce((maisCaro, p) => (p.preco > maisCaro.preco
? p : maisCaro), this.produtos[0]);
  }

  listarAbaixoDoMinimo(minimo) {
    return this.produtos.filter(p => p.quantidade < minimo);
  }
}

const estoque = new Estoque([
  { nome: "Teclado", preco: 250, quantidade: 5 },
  { nome: "Mouse", preco: 120, quantidade: 2 }
]);

console.log("Total em estoque:", estoque.calcularValorTotal());
```

Saída no console:

Total em estoque: 1490

Comparação:

Na versão anterior com array e funções isoladas, os dados ficavam separados dos métodos, exigindo que a lista fosse passada manualmente em cada chamada e ficando vulnerável a alterações externas no escopo.

Com a classe Estoque, os dados e as operações ficam agrupados na própria instância

(encapsulamento). O código fica mais limpo, fácil de reutilizar e mais organizado para dar manutenção.